

```

/*
=====
=====
    MEERA & ARJUN – WEDDING INVITATION
    script.js – loading sequence, mandala
    generator, scroll reveals,
    countdown, timeline fill, gallery
    lightbox, RSVP form, music toggle.
=====
===== */
'use strict';

/* -----
-----
    Small helpers
-----
----- */
const $ = (sel, ctx = document) =>
    ctx.querySelector(sel);
const $$ = (sel, ctx = document) =>
    Array.from(ctx.querySelectorAll(sel));

/*
=====
=====
    1. MANDALA GENERATOR
    Builds a symmetrical gold rosette by
    rotating one petal path
    around a center point. Reused for the
    loader, hero corners,

```

```

    couple divider, and footer.
    =====
    ===== */
    function buildMandala(svg, { petals = 12,
    radius = 100 } = {}) {
        if (!svg) return;
        const cx = 100, cy = 100;
        const ns = 'http://www.w3.org/2000/svg';

        // Outer + inner guide circles
        [radius * 0.92, radius * 0.62, radius *
    0.3].forEach((r) => {
            const circle =
    document.createElementNS(ns, 'circle');
            circle.setAttribute('cx', cx);
            circle.setAttribute('cy', cy);
            circle.setAttribute('r', r);
            svg.appendChild(circle);
        });

        // Petal path, repeated with rotation for
    n-fold symmetry
        const angleStep = 360 / petals;
        for (let i = 0; i < petals; i++) {
            const path =
    document.createElementNS(ns, 'path');
            const d = `M ${cx} ${cy - radius *
    0.28}
                C ${cx + radius * 0.22} ${cy
    - radius * 0.6}, ${cx + radius * 0.22} ${cy
    - radius * 0.82}, ${cx} ${cy - radius *
    0.92}
                C ${cx - radius * 0.22} ${cy
    - radius * 0.82}, ${cx - radius * 0.22}

```

```

    ${cy - radius * 0.6}, ${cx} ${cy - radius *
0.28} Z`;
    path.setAttribute('d', d);
    path.setAttribute('transform',
`rotate(${i * angleStep} ${cx} ${cy})`);
    svg.appendChild(path);
  }
  return svg;
}

// Build every mandala instance on the page
$('.mandala').forEach((svg) => {
  const isLoader =
svg.classList.contains('mandala--loader');
  buildMandala(svg, { petals: isLoader ? 16
: 12, radius: 92 });
});

/*
=====
=====
2. LOADING SCREEN
Fake a brief, elegant progress fill,
then reveal the page and
play the hero entrance.
=====
===== */
function runLoader() {
  const loader = $('#loader');
  const bar = $('#loaderProgress');
  let progress = 0;

  const tick = setInterval(() => {
    progress += Math.random() * 18 + 6;

```

```

    if (progress >= 100) {
      progress = 100;
      clearInterval(tick);
      bar.style.width = progress + '%';
      setTimeout(() => {
        loader.classList.add('is-hidden');
        document.body.style.overflow = '';
        playHeroEntrance();
      }, 350);
      return;
    }
    bar.style.width = progress + '%';
  }, 180);
}

// Lock scroll while loading
document.body.style.overflow = 'hidden';
window.addEventListener('load', () =>
  setTimeout(runLoader, 300));

/*
=====
=====
    3. HERO ENTRANCE + SCROLL REVEALS (GSAP
+ ScrollTrigger)
=====
===== */
function playHeroEntrance() {
  if (typeof gsap === 'undefined') return;
  gsap.to('[data-hero]', {
    opacity: 1,
    y: 0,
    duration: 1.1,
    ease: 'power3.out',

```

```

        stagger: 0.14,
    });
}

// Initial state for hero elements
(revealed by the timeline above)
gsap && gsap.set && gsap.set('[data-hero]',
{ opacity: 0, y: 28 });

if (typeof gsap !== 'undefined' && typeof
ScrollTrigger !== 'undefined') {
    gsap.registerPlugin(ScrollTrigger);

    $('[data-reveal]').forEach((el) => {
        ScrollTrigger.create({
            trigger: el,
            start: 'top 85%',
            onEnter: () => el.classList.add('is-
visible'),
            onEnterBack: () =>
el.classList.add('is-visible'),
        });
    });

    // Timeline connector line fills as the
section scrolls through view
    const timelineLine = $('#timelineFill');
    const timeline = $('.timeline');
    if (timelineLine && timeline) {
        ScrollTrigger.create({
            trigger: timeline,
            start: 'top 70%',
            end: 'bottom 60%',
            scrub: 0.6,

```

```

        onUpdate: (self) => {
            timelineLine.style.height =
            (self.progress * 100) + '%';
        },
    });
}
} else {
    // Fallback: reveal everything
    immediately if GSAP fails to load
    $$('[data-reveal]').forEach((el) =>
    el.classList.add('is-visible'));
}

/*
=====
=====

4. FALLING PETALS
=====
===== */
function spawnPetal() {
    const container = $('#petals');
    if (!container) return;
    const petal =
document.createElement('span');
    petal.className = 'petal';

    const left = Math.random() * 100;
    const duration = 9 + Math.random() * 8;
    const swayDuration = 3 + Math.random() *
3;
    const size = 7 + Math.random() * 8;
    const rotate = Math.random() * 360;

    petal.style.left = left + 'vw';

```

```

    petal.style.width = size + 'px';
    petal.style.height = size * 1.15 + 'px';
    petal.style.animationDuration =
`${duration}s, ${swayDuration}s`;
    petal.style.transform =
`rotate(${rotate}deg)`;
    petal.style.opacity = (0.35 +
Math.random() * 0.35).toFixed(2);

    container.appendChild(petal);
    setTimeout(() => petal.remove(), duration
* 1000);
}

// Gentle, capped petal stream – keeps
things ambient, not distracting
setInterval(spawnPetal, 900);
for (let i = 0; i < 6; i++)
setTimeout(spawnPetal, i * 300);

/*
=====
=====

5. STICKY HEADER
=====
===== */
const header = $('#siteHeader');
const hero = $('#hero');

function updateHeader() {
  if (!header || !hero) return;
  const heroBottom =
hero.getBoundingClientRect().bottom;
  header.classList.toggle('is-visible',

```

```

heroBottom < 80);
    header.classList.toggle('is-solid',
window.scrolly > 40);
}
window.addEventListener('scroll',
updateHeader, { passive: true });
updateHeader();

/*
=====
=====
6. MOBILE MENU
=====
===== */
const menuToggle = $('#menuToggle');
const menuOverlay = $('#menuOverlay');
const menuClose = $('#menuClose');

function openMenu() {
    menuOverlay.classList.add('is-open');
    menuOverlay.setAttribute('aria-hidden',
'false');
    menuToggle.setAttribute('aria-expanded',
'true');
}
function closeMenu() {
    menuOverlay.classList.remove('is-open');
    menuOverlay.setAttribute('aria-hidden',
'true');
    menuToggle.setAttribute('aria-expanded',
'false');
}

menuToggle &&

```



```

menuToggle.addEventListener('click',
openMenu);
menuClose &&
menuClose.addEventListener('click',
closeMenu);
$$('.menu-link').forEach((link) =>
link.addEventListener('click', closeMenu));

/*
=====
=====
7. COUNTDOWN TIMER
Set your real ceremony date/time below
(local time of the browser).
=====
===== */
const WEDDING_DATE = new Date('2026-12-
12T10:30:00');

function updateCountdown() {
  const now = new Date();
  let diff = WEDDING_DATE - now;
  if (diff < 0) diff = 0;

  const days = Math.floor(diff / (1000 * 60
* 60 * 24));
  const hours = Math.floor((diff / (1000 *
60 * 60)) % 24);
  const minutes = Math.floor((diff / (1000
* 60)) % 60);
  const seconds = Math.floor((diff / 1000)
% 60);

  setCountdownValue('cd-days', days);

```

```

    setCountdownValue('cd-hours', hours);
    setCountdownValue('cd-minutes', minutes);
    setCountdownValue('cd-seconds', seconds);
  }

function setCountdownValue(id, value) {
  const el = $('#'+ id);
  if (!el) return;
  const formatted =
String(value).padStart(2, '0');
  if (el.textContent !== formatted) {
    el.textContent = formatted;
    el.classList.remove('pulse');
    // Force reflow so the animation can
retrigger every second
    void el.offsetWidth;
    el.classList.add('pulse');
  }
}

updateCountdown();
setInterval(updateCountdown, 1000);

/*
=====
=====
      8. GALLERY LIGHTBOX
=====
===== */
const galleryItems = $('.gallery-item');
const lightbox = $('#lightbox');
const lightboxStage = $('#lightboxStage');
const lightboxCaption =
$('#lightboxCaption');

```

```
let currentIndex = 0;

function openLightbox(index) {
  currentIndex = index;
  renderLightbox();
  lightbox.classList.add('is-open');
  lightbox.setAttribute('aria-hidden',
    'false');
}
function closeLightbox() {
  lightbox.classList.remove('is-open');
  lightbox.setAttribute('aria-hidden',
    'true');
}
function renderLightbox() {
  const item = galleryItems[currentIndex];
  if (!item) return;
  // Reuse the gallery tile's gradient
  background for the large stage
  lightboxStage.style.background =
    getComputedStyle(item).backgroundImage;
  lightboxCaption.textContent =
    item.dataset.caption || '';
}
function showNext() { currentIndex =
  (currentIndex + 1) % galleryItems.length;
  renderLightbox(); }
function showPrev() { currentIndex =
  (currentIndex - 1 + galleryItems.length) %
  galleryItems.length; renderLightbox(); }

galleryItems.forEach((item, i) =>
  item.addEventListener('click', () =>
    openLightbox(i)));
```

```
$('#lightboxClose') &&
$('#lightboxClose').addEventListener('click'
, closeLightbox);
$('#lightboxNext') &&
$('#lightboxNext').addEventListener('click'
, showNext);
$('#lightboxPrev') &&
$('#lightboxPrev').addEventListener('click'
, showPrev);

lightbox &&
lightbox.addEventListener('click', (e) => {
if (e.target === lightbox) closeLightbox();
});

document.addEventListener('keydown', (e) =>
{
  if (!lightbox.classList.contains('is-
open')) return;
  if (e.key === 'Escape') closeLightbox();
  if (e.key === 'ArrowRight') showNext();
  if (e.key === 'ArrowLeft') showPrev();
});

// Basic swipe support for touch devices
let touchStartX = 0;
lightbox &&
lightbox.addEventListener('touchstart', (e)
=> { touchStartX =
e.changedTouches[0].screenX; }, { passive:
true });
lightbox &&
lightbox.addEventListener('touchend', (e)
=> {
```

```

    const delta = e.changedTouches[0].screenX
    - touchStartX;
    if (Math.abs(delta) > 40) delta > 0 ?
    showPrev() : showNext();
  }, { passive: true });

  /*
  =====
  =====
  9. RSVP FORM
  No backend is wired up here – connect
  this to your own endpoint
  (Formspree, EmailJS, Google Forms, etc.)
  by replacing the
  fakeSubmit() call below with a real
  fetch() request.
  =====
  ===== */
  const rsvpForm = $('#rsvpForm');
  const rsvpSubmit = $('#rsvpSubmit');
  const rsvpSuccess = $('#rsvpSuccess');
  const mealField = $('#mealField');
  const attendRadios =
    $('input[name="attending"]');

  function syncMealField() {
    const attending =
      $('input[name="attending"]:checked');
    if (!mealField) return;
    mealField.classList.toggle('is-hidden', !
      (attending && attending.value === 'yes'));
  }
  attendRadios.forEach((r) =>
    r.addEventListener('change',

```

```

syncMealField));
syncMealField();

function fakeSubmit() {
  // Simulated network delay – replace with
  a real request.
  return new Promise((resolve) =>
    setTimeout(resolve, 1200));
}

rsvpForm &&
rsvpForm.addEventListener('submit', async
(e) => {
  e.preventDefault();
  if (!rsvpForm.checkValidity()) {
    rsvpForm.reportValidity();
    return;
  }

  rsvpSubmit.classList.add('is-loading');
  rsvpSubmit.disabled = true;

  await fakeSubmit();

  rsvpForm.hidden = true;
  rsvpSuccess.hidden = false;
});

/*
=====
=====
10. BACKGROUND MUSIC
Autoplay is blocked by browsers until
the user interacts, so the

```

```

    button starts muted/paused and toggles
    on tap. Add your track at
    assets/music.mp3.
=====
===== */
const musicToggle = $('#musicToggle');
const bgMusic = $('#bgMusic');
let isPlaying = false;

musicToggle &&
musicToggle.addEventListener('click', async
() => {
  try {
    if (isPlaying) {
      bgMusic.pause();
    } else {
      await bgMusic.play();
    }
    isPlaying = !isPlaying;
    musicToggle.classList.toggle('is-
playing', isPlaying);
    musicToggle.setAttribute('aria-
pressed', String(isPlaying));
    musicToggle.setAttribute('aria-label',
isPlaying ? 'Pause background music' :
'Play background music');
  } catch (err) {
    // No audio file present, or playback
was blocked – fail silently in the UI.
    console.info('Background music
unavailable – add a track at
assets/music.mp3');
  }
}

```

```
}  
});
```